3.5 — Language Modelling (types, importance, curse of dimensionality, Markov assumption, N-grams) — 7 marks

Definition & goal:

A *language model (LM)* estimates the probability distribution of word sequences. Given a sequence $w_1, w_2, \ldots, w_n$, an LM assigns $P(w_1, w_2, \ldots, w_n)$ and can therefore predict next words or score text fluency. Synced | AI Tech...

Types of language models:

- **Count-based n-gram models** (probabilities from counts of contiguous n-word sequences).
- **Smoothed n-gram models** (Laplace, Good-Turing, Kneser-Ney) to handle unseen n-grams.
- **Neural (continuous-space) LMs** (feedforward, RNN/LSTM, Transformer) that learn dense representations. Stanford University +1

2. Types of Language Models (in Full Detail)

A) Count-based Models

These use *statistics from corpora*.

1. Unigram Model

Each word is independent:

$$P(W) = \prod_{i=1}^{n} P(w_i)$$

Very simple but unrealistic.

2. Bigram Model

Each word depends on only the previous word:

$$P(w_i|w_1^{i-1}) \approx P(w_i|w_{i-1})$$

3. Trigram / N-gram Models

A word depends on previous *n−1* words.

Example: trigram

$$P(w_i|w_{i-2}, w_{i-1})$$

B) Neural Language Models

These overcome curse of dimensionality using continuous vector embeddings.

1. Feedforward Neural LM (Bengio et al.)

Learns embeddings + predicts next word.

2. RNN / LSTM / GRU LM

Maintains hidden state → captures long-range dependencies.

3. Transformer-based LM (GPT, BERT, LLaMA, etc.)

Uses *self-attention* → best performance

Capable of long context understanding

⭐ LANGUAGE MODELLING — EXTENDED EXAM ANSWER (7 MARKS)

1. Definition of Language Modelling

Language Modelling (LM) is the task of assigning a probability to a sequence of words.

Given a sentence:

$$W = (w_1, w_2, \ldots, w_n)$$

a language model computes:

$$P(W) = P(w_1, w_2, \ldots, w_n)$$

A good LM assigns **higher probability to grammatically correct, natural sentences** and lower probability to unnatural sentences.

Example:

- "The cat sat on the mat" → high probability
- "Mat the on sat cat the" → low probability

★ 2. Types of Language Modelling

Language models evolved from simple count-based models to deep learning models.

A) Count-Based Statistical Models

These rely on word frequencies in a corpus.

(i) Unigram Model

Assumes each word is independent:

$$P(W) = \prod_{i=1}^{n} P(w_i)$$

Unrealistic but simple.

(ii) Bigram Model

Word depends only on previous word:

$$P(w_i|w_1^{i-1}) \approx P(w_i|w_{i-1})$$

(iii) Trigram / N-gram Models

A word depends on previous **n−1** words:

$$P(w_i|w_1^{i-1}) \approx P(w_i|w_{i-n+1}^{i-1})$$

These models are computed using counts:

$$P(w_n|context) = \frac{C(context, w_n)}{C(context)}$$

B) Neural Language Models

Overcome limitations of statistical models.

(i) Feedforward Neural LM

Introduced by Bengio et al.

Learns embeddings + predicts next word.

(ii) Recurrent Neural Networks (RNN, LSTM, GRU)

Processes sequence one word at a time, maintains memory.

(iii) Transformer-Based Models

Models like **BERT, GPT, LLaMA** use self-attention.

Advantages:

- Long context understanding
- State-of-the-art results
- Parallel training

⭐ 3. Importance of Language Modelling (Expanded)

Language modelling is foundational to almost every NLP task:

1. Text Prediction & Auto-complete

Predict next word in mobile typing or search bars.

2. Machine Translation

LM checks fluency of output sentences.

3. Speech Recognition

When multiple words sound similar, LM selects the most likely one.

4. Spelling Correction & Auto-correct

Chooses correction with the highest probability.

5. Chatbots and Dialogue Systems

Ensures grammatically correct responses.

6. Text Generation

Used in:

- ChatGPT
- Story generators
- Paraphrasers

7. Information Retrieval

Improves ranking of search results.

⭐ 4. The Curse of Dimensionality (FULL EXPLANATION)

Statistical LMs suffer from huge data requirements because:

✔ Vocabulary size (V) is large

✔ N-gram combinations grow exponentially

For example, with V = 10,000:

- Bigram space = $10^4 \times 10^4 = 10^8$
- Trigram space = 10^{12}

Problems caused:

1. **Data sparsity**

   Most N-grams never occur → probability becomes zero.
2. **Memory explosion**

   Need to store huge count tables.
3. **Poor generalization**

   Cannot handle unseen phrases.
4. **High computational cost** for estimating probabilities.

Neural LMs solve this problem using **embeddings** and continuous vector spaces.

⭐ 5. Markov Assumption in Language Modelling

The **Markov Assumption** simplifies sequence probability by assuming:

👉 A word depends only on a limited number of previous words.

General form:

$$P(w_i|w_1^{i-1}) \approx P(w_i|w_{i-n+1}^{i-1})$$

Why we need this assumption:

- Reduces complexity
- Makes probability computation feasible
- Supports N-gram model creation

⭐ 6. N-Gram Models (FULL EXPLANATION)

An N-gram is a sequence of **N consecutive words**.

Examples:

- Unigram: N=1 → "love"
- Bigram: N=2 → ("I", "love")
- Trigram: N=3 → ("I", "love", "NLP")

Probability Computation (Maximum Likelihood Estimate):

$$P(w_n|w_1^{n-1}) \approx \frac{C(w_1^n)}{C(w_1^{n-1})}$$

Problems

- Zero counts → probability = 0

  Solution: **Smoothing Techniques:**
- Laplace Smoothing
- Lidstone Smoothing
- Good-Turing
- Kneser-Ney Backoff (best traditional m↓d)

⭐ LANGUAGE MODEL IMPLEMENTATION — FULL EXTENDED NOTES

Language Model Implementation involves building an **N-gram statistical language model** from scratch. The model estimates the probability of each word given its context and uses this to score or generate text.

🔶 1. Setup for Language Model Implementation

Before building an LM, several preparation steps are required:

✔ 1. Load the training corpus

- Can be plain text files (Wikipedia, news, books)
- Large corpus improves coverage and accuracy

✔ 2. Preprocessing steps

- Convert text to lowercase
- Remove or normalize punctuation
- Normalize numbers and dates
- Split text into sentences
- Tokenize sentences into words
- Replace rare words with `<UNK>` (unknown token)
- Add sentence boundary tokens such as:
  - `<s>` for start of sentence
  - `</s>` for end of sentence

✔ 3. Choose model order (n-gram)

- Bigram (n=2)
- Trigram (n=3)
- Higher-order N-grams capture more context but increase sparsity

✔ 4. Data structures needed

- A dictionary for N-gram counts
- A dictionary for (N-1)-gram context cou

2. N-grams Function (Extracting n-word sequences)

An **N-gram** is a sequence of N consecutive tokens.

Example sentence:

"the cat sat on mat"

- Bigrams: (the, cat), (cat, sat), (sat, on), (on, mat)
- Trigrams: (the, cat, sat), (cat, sat, on), (sat, on, mat)

Purpose of N-grams Function

- Sliding window over tokens to generate N-gram tuples
- Used for building LM counts and computing probability

Algorithm (step-by-step)

sql Copy code

```
for each sentence:
    add <s> tokens at start, </s> at end
    for i from N-1 to len(tokens)-1:
        ngram = tokens[i-N+1 ... i]
        yield ngram
```

This function ensures:

- All n-grams are extracted
- Model computes context → next word relations

🔶 3. Update Counts Function (Building frequency tables)

Language models rely on frequencies of N-grams.

We maintain two tables:

✔ **N-gram count:**

$$C(w_{i-n+1}, \ldots, w_i)$$

✔ **(N-1)-gram context count:**

$$C(w_{i-n+1}, \ldots, w_{i-1})$$

Algorithm

Copy code

```
counts[ngram] += 1
context_counts[context] += 1
```

Why context counts are needed

To compute conditional probability:

$$P(w_i|context) = \frac{C(context, w_i)}{C(context)}$$

If context count is 0 → probability cannot be computed → requires smoothing.

4. Probability Model Function (Computing LM probabilities)

The LM predicts the next word using the conditional probability:

$$P(w_i|w_{i-n+1}^{i-1}) = \frac{C(w_{i-n+1}^{i})}{C(w_{i-n+1}^{i-1})}$$

Problems

- For unseen N-grams → count = 0
- Probability becomes zero
- Entire sentence probability becomes zero

Solution: Smoothing

✔ Add-One Smoothing (Laplace)

$$P = \frac{C(ngram) + 1}{C(context) + V}$$

Where **V** = vocabulary size.

✔ Add-k Smoothing

$$P = \frac{C + k}{context + kV}$$

✔ Kneser-Ney Smoothing (best)

- Discount frequencies
- Backoff to lower-order N-grams
- Produces best performance for N-gram models

Sentence probability

$$P(sentence) = \prod_i P(w_i|context)$$

Log probability (to avoid underflow)

$$\log P(sentence) = \sum_i \log P(w_i|context)$$

Log probabilities are essential for long sentences.

🔶 5. Reading the Corpus (Full Explanation)

Reading and cleaning the corpus is crucial for building a robust LM.

Steps involved:

✔ Sentence Segmentation

Break text into sentences using:

- Periods, question marks, exclamation marks
- NLP tools (NLTK, spaCy)

✔ Tokenization

Split sentences into words handling:

- Contractions
- Punctuation
- Hyphens
- Apostrophes

✔ Normalization

- lowercase
- replace URLs, emails, numbers
- remove extra whitespace

✔ Rare Word Handling

Replace infrequent words with `<UNK>` to reduce vocabulary size and sparsity.

✔ Padding Tokens

Add:

- `<s>` repeated (n-1) times at beginning
- `</s>` at end

  This ensures first word has a valid context and model can generate endings.

🔶 6. Sampling Text from Language Model (Text Generation)

Sampling means generating text using the learned n-gram probabilities.

Algorithm for sampling:

Step 1 — Start with context

Begin with n-1 `<s>` tokens.

Example for trigram:

`["<s>", "<s>"]`

Step 2 — Predict next word

Use probability model to compute:

$$P(w|context)$$

Step 3 — Choose a word

- Greedy sampling → choose max probability
- Random sampling → choose based on distribution
- Temperature sampling → control randomness
- Top-k, top-p sampling → limit search space (used in GPT models)

Step 4 — Update context

Append new word and slide the window.

Step 5 — Stop when `</s>` is produced

Ensures natural sentence boundaries.

⭐ 7. COMPLETE PIPELINE SUMMARY (Write this at end of exam)

1. **Setup:** load corpus, preprocess, tokenize, add `<s>` and `</s>`.
2. **Extract N-grams:** using sliding window function.
3. **Update Counts:** store N-gram and context counts in dictionaries.
4. **Probability Model:** compute smoothed conditional probabilities and log-probabilities.
5. **Sentence Scoring:** multiply or sum log probabilities.
6. **Sampling Text:** generate new sentences using learned distribution.

This completes a full statistical N-gram language model implementation.

⭐ MARKOV MODELS — EXTENDED EXAM ANSWER (7–10 marks)

Markov Models are essential statistical tools in NLP for modeling sequences such as words, characters, and tags. They model the probability of future events based only on a limited history, not the entire sequence.

🔶 1. Markov Property (Deep Explanation)

The **Markov Property** states that:

> *The future state depends only on the present state, not on the full sequence of past states.*

Mathematically, for a sequence $X_1, X_2, ..., X_n$:

$$P(X_{n+1}|X_1, X_2, ..., X_n) = P(X_{n+1}|X_n)$$

This is **first-order Markov assumption**.

Higher-order Markov property:

For an n-th order Markov process:

$$P(X_{t+1}|X_1, ..., X_t) = P(X_{t+1}|X_{t-n+1}, ..., X_t)$$

2. Markov Model (Detailed Explanation)

A **Markov Model** is a stochastic model that uses transition probabilities between states.

In NLP, the "states" can be:

- Words
- Characters
- Part-of-speech tags
- Hidden states (in HMM)

Example: Bigram Markov Model

States: words

Transition probability:

$$P(w_i|w_{i-1})$$

General N-gram Markov model:

$$P(w_i|w_{i-n+1}^{i-1})$$

Components of a Markov Model:

1. **Set of states**
2. **Transition probabilities** between states
3. **Initial probability distribution**

🔶 3. Probability Smoothing (Very Important Topic)

Why Smoothing?

When a sequence has **zero count** in the training corpus, the probability becomes zero:

$$P(w_i|w_{i-1}) = \frac{0}{C(w_{i-1})} = 0$$

This is a problem because:

- One zero term → entire sentence probability becomes zero
- Unrealistic in natural language

Types of Smoothing:

A) Add-One Smoothing (Laplace)

Adds 1 to all counts:

$$P(w_i|w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}$$

Where **V** = vocabulary size.

B) Add-k Smoothing

Adds a small value k instead of 1 (0 < k < 1):

$$P = \frac{C + k}{N + kV}$$

C) Good-Turing Smoothing

Re-estimates counts of unseen events by redistributing probability mass from frequent events.

D) Backoff and Interpolation

Used in **Kneser-Ney Smoothing**, the best method.

Backoff:

- If trigram is unseen → use bigram
- If bigram unseen → use unigram

Interpolation:

- Combine all levels:

$$P = \lambda_1 P_{3gram} + \lambda_2 P_{2gram} + \lambda_3 P_{1gram}$$

🔶 4. Log Probabilities (Critical for Implementation)

Probabilities of sentences are extremely small (e.g., 10^{-300}).
Multiplying many small numbers → leads to **underflow**.

Solution: Use log probabilities.

$$\log P(w_1^n) = \sum_{i=1}^{n} \log P(w_i|context)$$

Advantages:

- Prevents underflow
- Turns multiplication into addition → faster computation
- Makes scoring long sequences stable

Example:
Instead of

$$P = P_1 \times P_2 \times P_3$$

Use

$$\log P = \log P_1 + \log P_2 + \log P_3$$

🔶 5. Building a Text Classifier Using Markov Models (Full Details)

Markov Models can classify texts based on likelihood under different language models.

Step 1: Build One LM per Category

Example: spam vs ham

- Train **bigram LM** on spam emails
- Train **bigram LM** on ham emails

Each LM learns its own transition probabilities.

Step 2: Compute Sentence Likelihood

Given an incoming email E:

$$Score_{spam}(E) = \log P(E|LM_{spam})$$

$$Score_{ham}(E) = \log P(E|LM_{ham})$$

Whichever score is higher → assign label.

Step 3: Decision Rule

$$\hat{y} = \arg\max_{class} \log P(E|LM_{class})$$

This is called a **generative classifier** because each class generates text using its Markov transitions.

Strengths of Markov Text Classifier:

- Simple
- Fast
- Interpretable
- Works well on structured text (e.g., aut' ↓ hip attribution, spam detection)

⭐ ARTICLE SPINNING — FULL DETAILED ANSWER

1. Problem description (what is article spinning)

Article spinning (text rewriting) is the automatic generation of alternative surface forms of the same content—keeping meaning while changing wording. Goal: produce readable, grammatically fluent, and semantically equivalent variants of a sentence or paragraph.

Uses: SEO content variation, paraphrase generation, data augmentation.
Challenges: preserve meaning, avoid grammatical errors, maintain cohesion, avoid nonsensical substitutions, and avoid ethical/copyright misuse.

---

2. N-gram approach — intuition and math

Idea: generate candidate rewrites by substituting words/phrases with synonyms or alternatives, then use a language model (LM) to *score fluency*. Select candidates that maximize LM probability while respecting semantic constraints.

Notation & scoring:

- Sentence $S = w_1 \dots w_m$.
- Candidate sentence S' formed by substituting tokens at some positions.
- Use an n-gram LM to score sentence probability:

$$\log P(S') = \sum_{i=1}^{m} \log P(w'_i \mid w_{i-n+1}^{i-1'})$$

Use **log-probs** to avoid underflow and make sums. Higher log-prob = more fluent.

Smoothing: use add-k (Laplace/add-ε) or better methods (Kneser-Ney). For add-k:

$$P(w_i \mid context) = \frac{C(context, w_i) + k}{C(context) + k \cdot V}$$

where V is vocabulary size, k small (e.g., 0.1).

Search strategy: naive enumeration explodes combinatorially. Use **beam search** (keep top-B candidates at each substitution step) or greedy local replacements with LM scoring.

Semantic constraints: restrict candidate set to same POS, synonyms with similar

$$\log P(S') = \sum_{i=1} \log P(w'_i \mid w'^{i-1}_{i-n+1})$$

Use **log-probs** to avoid underflow and make sums. Higher log-prob = more fluent.

Smoothing: use add-k (Laplace/add-ε) or better methods (Kneser-Ney). For add-k:

$$P(w_i \mid context) = \frac{C(context, w_i) + k}{C(context) + k \cdot V}$$

where V is vocabulary size, k small (e.g., 0.1).

Search strategy: naive enumeration explodes combinatorially. Use **beam search** (keep top-B candidates at each substitution step) or greedy local replacements with LM scoring.

Semantic constraints: restrict candidate set to same POS, synonyms with similar embeddings, or paraphrase database (PPDB) to preserve meaning.

⭐ Cipher Decryption with Language Modeling and Genetic Algorithm

(Ciphers, Substitution Cipher, Bigrams, Maximum & Log-Likelihood, Language Models, Genetic Algorithms)

You can WRITE THIS EXACTLY in your exam to score full marks.
The explanation is clear, mathematical, complete, and follows university-level NLP syllabus.

⭐ 1. CIPHERS (INTRODUCTION)

A **cipher** is a method of transforming plaintext into ciphertext to protect information. The transformation is governed by a *key*.

Two main types:

1. **Transposition ciphers** – rearrange letters, keep letters unchanged.
2. **Substitution ciphers** – replace each letter with another letter.

In NLP-based cryptanalysis, we mainly work with **monoalphabetic substitution ciphers** because their structure preserves word length but scrambles letter distribution.

⭐ 2. SUBSTITUTION CIPHER (FULL EXPLANATION)

A **substitution cipher** replaces each letter in plaintext with a *fixed* corresponding letter.

Example mapping:
A→M, B→Q, C→R, … Z→G

Plaintext: **THE CAT**
Ciphertext: **ZPM RMR**

Properties:

- Key space = 26! ≈ 4×10^{26} (IMPOSSIBLE to brute force)
- Same pattern appears: double letters, common digrams (“TH”, “HE”).
- Frequency of letters preserved, but permuted.

⭐ 2. SUBSTITUTION CIPHER (FULL EXPLANATION)

A **substitution cipher** replaces each letter in plaintext with a *fixed* corresponding letter.

Example mapping:

A→M, B→Q, C→R, … Z→G

Plaintext: **THE CAT**

Ciphertext: **ZPM RMR**

Properties:

- Key space = 26! ≈ 4×10^{26} (IMPOSSIBLE to brute force)
- Same pattern appears: double letters, common digrams (“TH”, “HE”).
- Frequency of letters preserved, but permuted.

Goal of decryption:

👉 **Recover the key (mapping)** without knowing plaintext.

We use **statistics of English + Language Models + Genetic Algorithms**.

⭐ 3. BIGRAMS (CHARACTER N-GRAMS FOR SCORING)

A **bigram** is a pair of consecutive characters.

Example:

“hello” → he, el, ll, lo

Why bigrams?

- English has strong character patterns: “TH”, “HE”, “IN”, “ER”.
- Rare bigrams (ZX, JQ) are unlikely.
- Therefore, scoring a decrypted sentence by summing **bigram probabilities** tells us how “English-like” the text is.

Probability of a string under a bigram LM:

$$P(w_1 w_2 \dots w_n) = \prod_{i=2}^{n} P(w_i \mid w_{i-1})$$

Better computed in log-space.

⭐ 4. MAXIMUM LIKELIHOOD & LOG-LIKELIHOOD

Goal:

Find the key **K** that makes decrypted text **most probable** under English language statistics.

Let C = ciphertext, D = decryption using key K.

Maximum Likelihood Estimation (MLE):

$$K^* = \arg\max_K P(D \mid LM)$$

Because probabilities are extremely small, we use **log-likelihood**:

$$LL(D) = \sum_{i=2}^{n} \log P(d_i \mid d_{i-1})$$

We choose key with **maximum log-likelihood**.

Why logs?

- Avoid underflow
- Convert multiplication → addition
- Allows easy comparison

⭐ 5. LANGUAGE MODELS FOR CRYPTANALYSIS

A **language model (LM)** gives probabilities for sequences of letters or words.

For substitution cipher decryption, we use **character-level LM**:

Types:

- Unigram (single letters)
- **Bigram (pairs of letters)**
- Trigram

Bigram LM is most commonly used because:

- Strong English patterns
- Fast to compute
- Good balance between accuracy & efficiency

LM Construction:

1. Take a large English corpus
2. Count occurrences of each bigram
3. Estimate probabilities with smoothing:

$$P(b_i \mid b_{i-1}) = \frac{count(b_{i-1}, b_i) + k}{count(b_{i-1}) + kV}$$

(Laplace/add-k smoothing)

⭐ 6. WHY GENETIC ALGORITHMS FOR DECRYPTION?

The key space for substitution cipher = **26! ≈ 4 × 10²⁶**

→ brute force impossible.

GA is used because:

- Efficient for huge search space
- Finds near-optimal key quickly
- Uses probabilistic scoring (log-likelihood)

⭐ 7. GENETIC ALGORITHM (FULL WORKFLOW)

A **Genetic Algorithm (GA)** is a heuristic inspired by biological evolution.

GA components for cipher decryption:

A) Representation (Chromosome / Individual)

A chromosome = **a permutation of 26 letters** representing key mapping.

Example:
[Q,W,E,R,T,Y,U,I,O,P,A,S,D,F,G,H,J,K,L,Z,X,C,V,B,N,M]

B) Initial Population

Generate N random permutations of alphabet.

C) Fitness Function

Fitness = **log-likelihood of decrypted text** under bigram LM.

$$Fitness(K) = LL(decrypt(C, K))$$

High score = more English-like → better key.

D) Selection

Choose best scoring individuals for reproduction.

Methods:

- Tournament selection
- Roulette-wheel selection
- Rank-based selection

E) Crossover (Recombination)

Combine two parents to create offspring.

Common method: **cycle crossover** (CX) to preserve permutation constraints.

F) Mutation

Randomly swap two letters in key.

Purpose:

- Explore new areas of search space
- Prevent premature convergence

G) Replacement

Population updated using:

- Elitism (keep best)
- Offspring replace worst performers

H) Termination

Stop when:

- Log-likelihood stops improving
- Max generations reached
- Decryption is readable English

⭐ 8. OVERALL PIPELINE (Full Summary)

Step 1: Input ciphertext.

Step 2: Train a bigram LM from English corpus.

Step 3: Initialize random population of keys.

Step 4: For each key:

Decrypt → Compute log-likelihood → Assign fitness.

Step 5: GA operations:

Selection → Crossover → Mutation → New keys

Step 6: Repeat until best key stabilizes.

Step 7: Output best key & decrypted text.

This method can decrypt long ciphertexts with **high accuracy**.

⭐ 9. EXAM CONCLUSION (WRITE THIS AT END)

A substitution cipher can be cracked using statistical NLP by combining a **character bigram language model** with a **genetic algorithm**. The LM provides the scoring (maximum log-likelihood) that guides the GA search in the enormous key space. GA operations—selection, crossover, and mutation—efficiently evolve the key until the decrypted text matches natural English. This hybrid NLP-evolutionary approach is one of the most effective techniques for classical cipher decryption.